

Software Design Specification for

CampusNexus

A Campus-Centric Rental & Community Services Platform

CSE 3210: Software Project LAB (SPL)

B.Sc. in Computer Science and Engineering

SUBMITTED BY

Name	ID	Batch / Year
Abdullah Al Noman	12208004	2021-22

SUPERVISED BY

Md. Zahidur Rahman

Lecturer,

Department of CSE, Comilla University

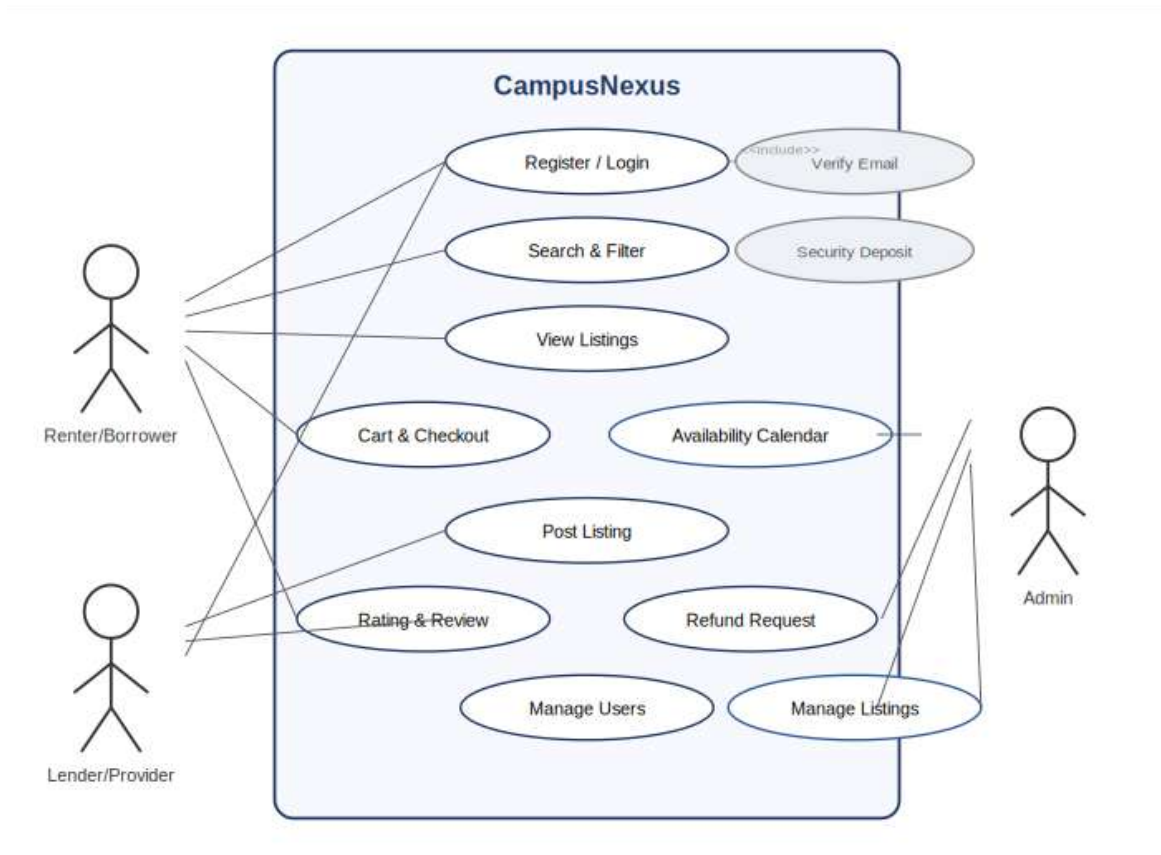
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COMILLA UNIVERSITY**

Submission Date: 10 May 2026

UML Diagrams

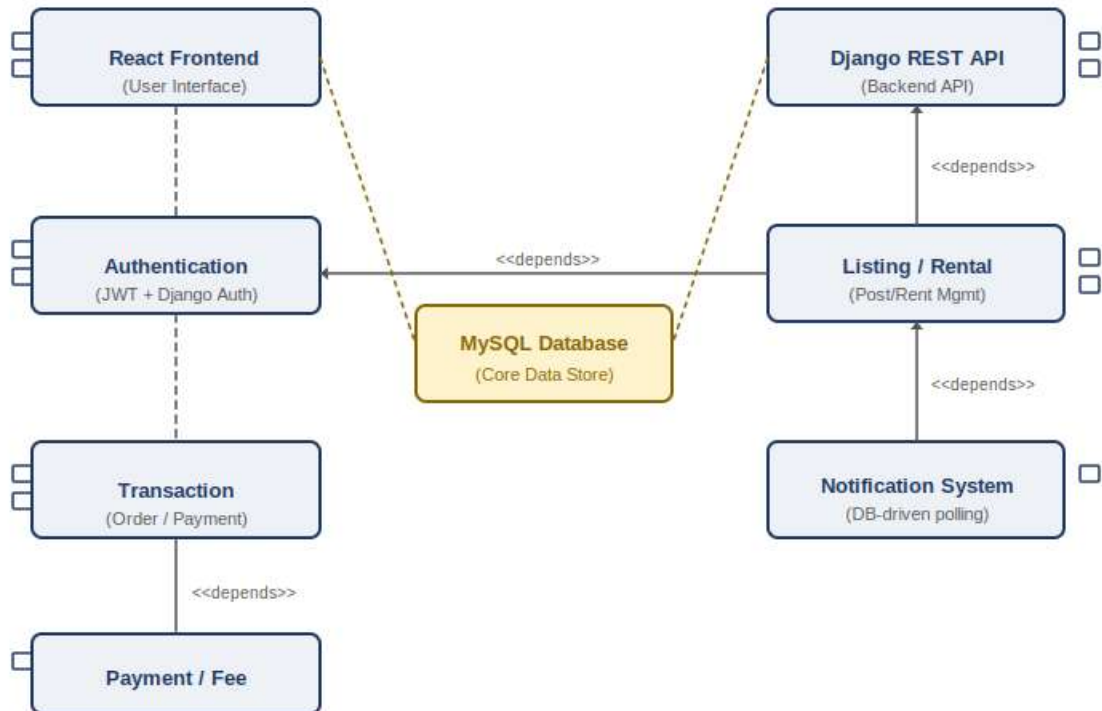
Use-Case Diagram

The use-case diagram shows the interactions between three actors — Renter/Borrower, Lender/Provider, and Admin — and the core features of CampusNexus including Registration, Search, Cart, Availability Calendar, Listings, Rating & Review, Refund, and Admin management.



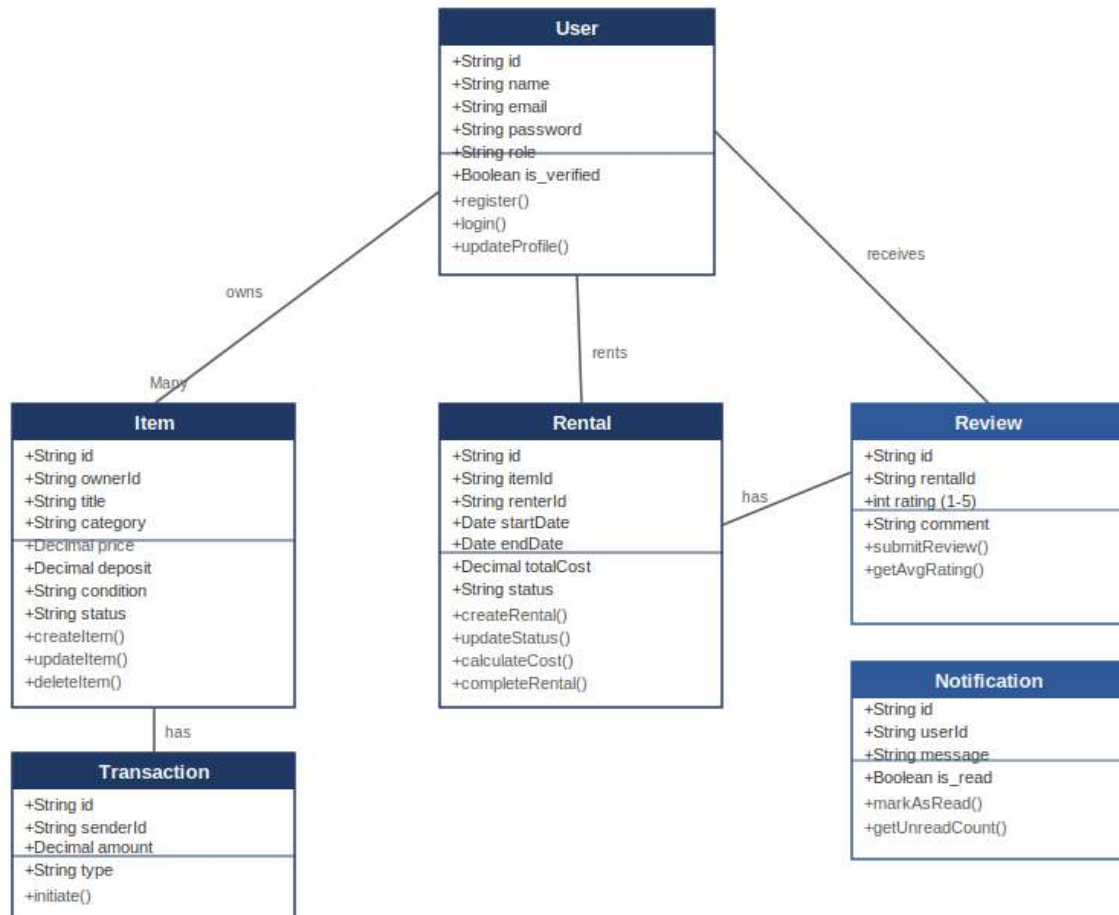
Component Diagram

The component diagram illustrates the architectural components of CampusNexus: React Frontend, Authentication Service (JWT), Django REST API, Listing/Rental Management, Transaction module, Payment/Fee module, Notification System, and the central MySQL Database. All components ultimately depend on or interact with the MySQL database as the core data store.



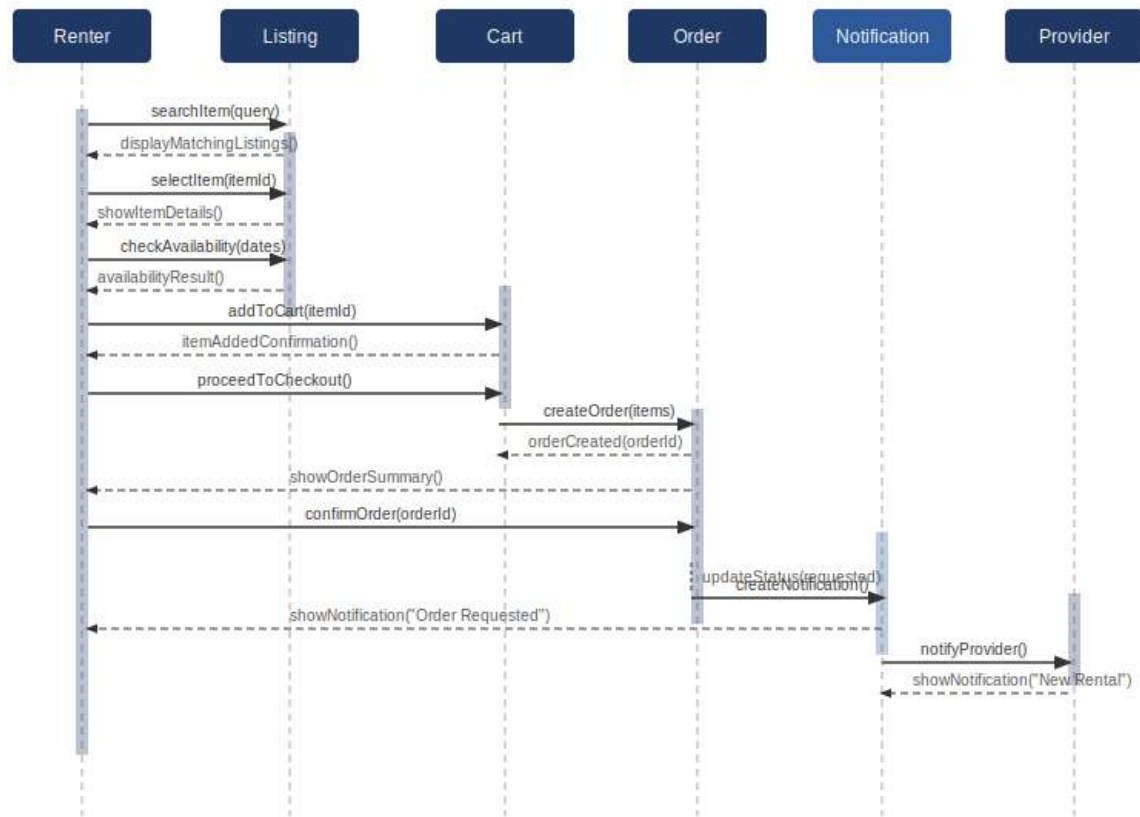
Class Diagram

The class diagram defines the six core classes of CampusNexus — User, Item, Rental, Transaction, Review, and Notification — with their attributes, methods, and relationships. Key relationships include: User owns many Items, User rents many Rentals, Rental has many Reviews, and User receives many Notifications.



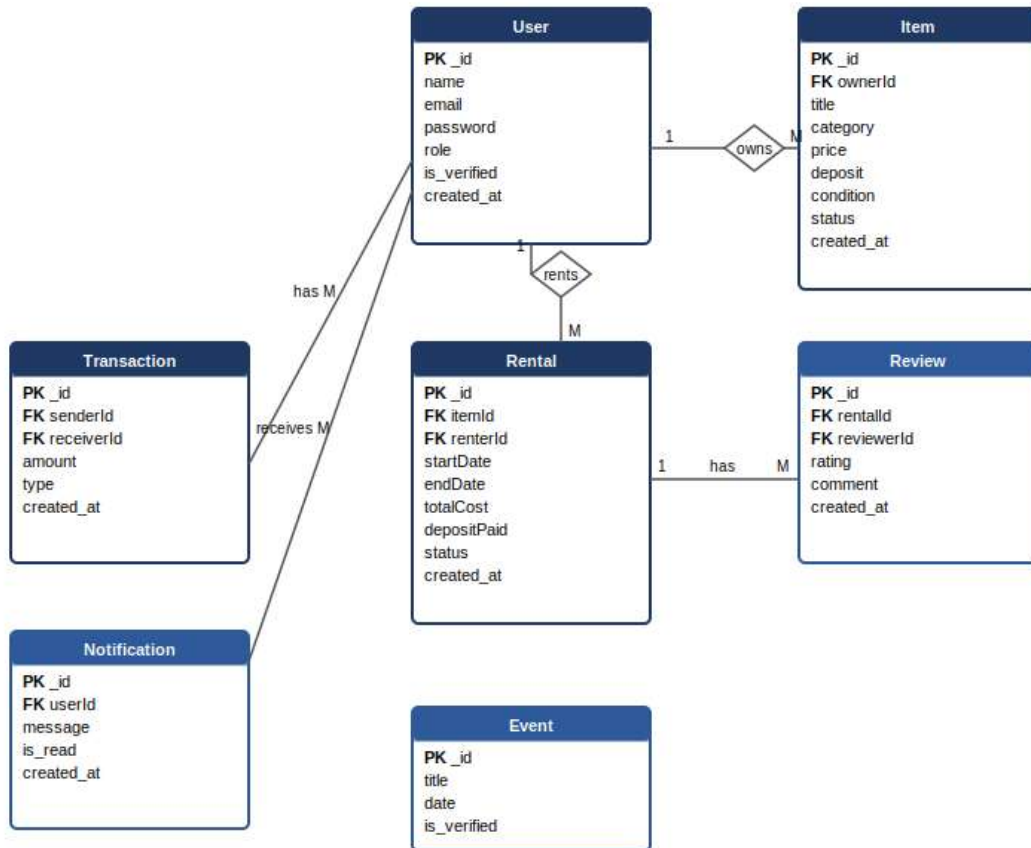
Sequence Diagram

The sequence diagram illustrates the full rental flow from a Renter's perspective: searching for items, checking availability, adding to cart, proceeding to checkout, confirming the order, and receiving database-driven notifications. The Provider is also notified when a new rental request is placed.



ER Diagram

The Entity-Relationship (ER) diagram shows the full database schema for CampusNexus with six core entities: User, Item, Rental, Transaction, Review, Notification, and Event. Key relationships: User (1) owns (M) Item, User (1) rents (M) Rental, Rental (1) has (M) Review, User (1) receives (M) Notification, and User (1) has (M) Transaction.

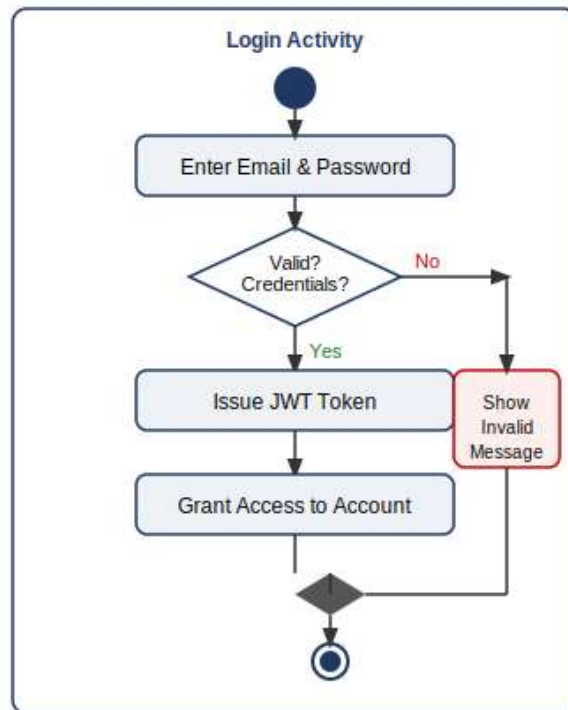


Activity Diagrams

The following activity diagrams illustrate the key workflows in CampusNexus:

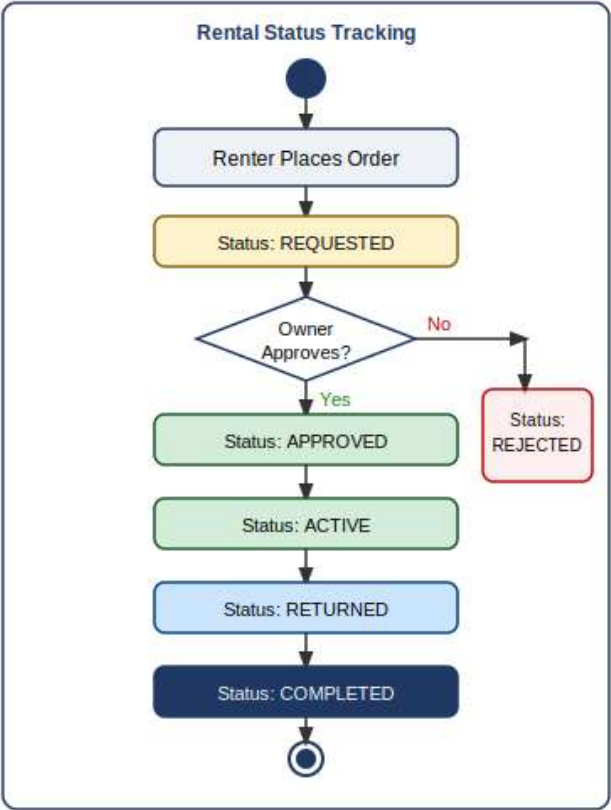
Activity Diagram 1: Login Workflow

Shows the login process — user enters credentials, system validates, issues JWT token on success, or shows invalid credentials message on failure.



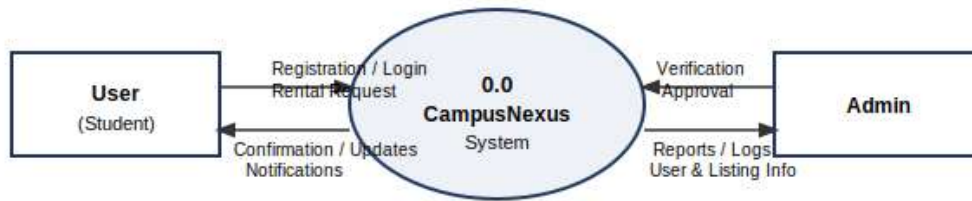
Activity Diagram 2: Rental Status Tracking Workflow

Illustrates the full rental lifecycle — from order placement (REQUESTED) through owner approval (APPROVED), start date (ACTIVE), item return (RETURNED), to final completion (COMPLETED). A rejection path is also shown.



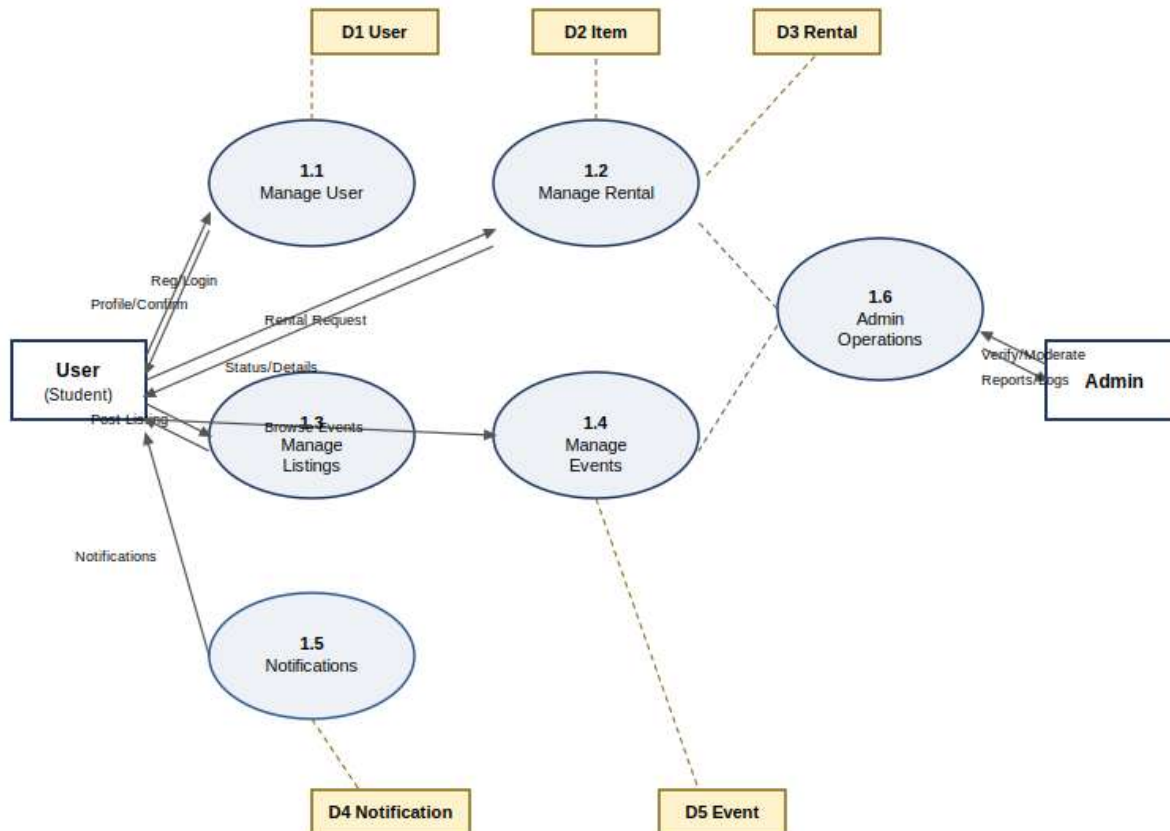
DFD Level 0 (Context Diagram)

The DFD Level 0 shows CampusNexus as a single system process (0.0) interacting with two external entities: User (Student) and Admin. Users send registration/login info and rental requests; the system returns confirmations, updates, and notifications. Admins provide verification and approval; the system returns reports and system logs.



DFD Level 1

The DFD Level 1 decomposes the CampusNexus system into six sub-processes: 1.1 Manage User Information, 1.2 Manage Rental, 1.3 Manage Listings, 1.4 Manage Events, 1.5 Notification System, and 1.6 Admin Operations. Five data stores are shown: D1 User, D2 Item, D3 Rental, D4 Notification, and D5 Event.



Design Decisions & Justification

Why Django + React + MySQL?

The technology stack is chosen based on the developer's existing skills and the database-focused nature of this project:

- Django provides a clean ORM for MySQL, built-in admin panel, and REST API support via DRF — ideal for a database-heavy project.
- React provides a component-based frontend that efficiently renders dynamic data from MySQL through REST API calls.
- MySQL is the core focus of this project — 11 relational tables with complex JOINS, aggregations, status workflows, and transactions.

Why No WebSocket or Third-Party Payment?

- The notification system uses database polling (MySQL Notification table) instead of WebSocket/Redis — simpler, no extra services, and still demonstrates database operations.
- Payment is simulated (demo flow) to avoid external API dependency while still demonstrating Transaction table operations.

Rental Status as a State Machine

The Rental table's status field implements a state machine: REQUESTED → APPROVED → ACTIVE → RETURNED → COMPLETED (or REJECTED / OVERDUE). This pattern demonstrates complex conditional database UPDATE operations and event-driven notifications, directly fulfilling CLO 7.

— End of Software Design Specification: CampusNexus —